开发者社区

DTK

DTK 26.10
hipSPARSE 库使用手册

DTK-261000-030-0915-945

目　录

1 hipSPARSE 库简介 1
2 环境要求 1
3 核心概念 1
3.1 API 使用说明 1
3.2 向量格式 2
3.2.1 密集向量格式 2
3.2.2 稀疏向量格式 2
3.3 矩阵格式 3
3.3.1 密集矩阵格式 3
3.3.2 坐标格式（COO） 3
3.3.3 压缩稀疏行格式（CSR） 4
3.3.4 压缩稀疏列格式（CSC） 5
3.3.5 块稀疏行格式（BSR） 6
3.3.6 切片稀疏格式（SELL） 7
3.4 简单 demo 8
4 API 详解 13
4.1 hipSPARSE 辅助函数 13
4.1.1 hipsparseCreate 13
4.1.2 hipsparseDestroy 13
4.1.3 hipsparseGetErrorName 13
4.1.4 hipsparseGetErrorString 14
4.1.5 hipsparseGetVersion 15
4.1.6 hipsparseGetProperty 15
4.1.7 hipsparseSetStream 15
4.1.8 hipsparseGetStream 16
4.1.9 hipsparseSetPointerMode 16
4.1.10 hipsparseGetPointerMode 17
4.1.11 hipsparseCreateMatDescr 17

开发者社区

4.1.12 hipsparseDestroyMatDescr 17
4.1.13 hipsparseCopyMatDescr 18
4.1.14 hipsparseSetMatType 18
4.1.15 hipsparseGetMatType 19
4.1.16 hipsparseSetMatFillMode 19
4.1.17 hipsparseGetMatFillMode 19
4.1.18 hipsparseSetMatDiagType 20
4.1.19 hipsparseGetMatDiagType 20
4.1.20 hipsparseSetMatIndexBase 20
4.1.21 hipsparseGetMatIndexBase 21
4.1.22 hipsparseCreateHybMat 21
4.1.23 hipsparseDestroyHybMat 21
4.2 level 1 函数 22
4.2.1 hipsparseXaxpyi 22
4.2.2 hipsparseXdoti 22
4.2.3 hipsparseXdotci 23
4.2.4 hipsparseXgthr 24
4.2.5 hipsparseXgthrz 24
4.2.6 hipsparseXroti() 25
4.2.7 hipsparseXsctr() 26
4.3 level 2 函数 26
4.3.1 hipsparseXcsrmv() 26
4.3.2 hipsparseXhybmv 27
4.3.3 hipsparseXbsrmv 28
4.3.4 hipsparseXbsrxmv() 29
4.3.5 hipsparseXbsrsv2_solve 30
4.3.6 hipsparseXgemvi 31
4.4 Level 3 函数 32
4.4.1 hipsparseXbsrmm 32
4.4.2 hipsparseXcsrmm 33

4.4.3 hipsparseXgemmi 34
4.5 预处理器函数 35
4.5.1 hipsparseXbsrilu02 35
4.5.2 hipsparseXcsrilu02 36
4.5.3 hipsparseXbsric02 37
4.5.4 hipsparseXcsric02 38
4.5.5 hipsparseXgtsv2 39
5 使用指南 39
5.1 设备管理 39
5.2 流管理 40
5.3 异步执行 40
5.4 单元测试 40

开发者社区

版本更新：

- DTK-25.04.1：
  - 创建文档。
- DTK-26.10：

  hipsparseAxpby、hipsparseSpvv、hipsparseSpmm、hipsparseSddmm 增加 half float 混合精度支持，其中 AB 使用 float16，C 和计算类型使用 float。

  hipsparseAxpby、hipsparseSpvv、hipsparseSpmv、hipsparseSpmm、hipsparseSddmm 增加 brain half float 混合精度支持，其中 AB 使用 bfloat16，C 和计算类型使用 float。

  hipsparseSddmm 支持 ABC 使用 bfloat16，计算类型使用 float。

  hipsparseScatter、hipsparseGather、hipsparseSddmm 支持 half float 类型和 brain half float 类型。

  hipsparseCsr2cscEx2 支持 int8 类型。

  spmv 新增性能更优越的确定性算法 flat 替换了原本的 stream 算法，用户可以在运行 spmv 接口时使用 HIPSPARSE_SPMV_CSR_ALG2 选项调用。

开发者社区

1 hipSPARSE 库简介

hipSPARSE 是 DTK 中的一套 GPU 加速的基本线性代数子程序，用于处理稀疏矩阵，其执行速度远快于仅使用 CPU 的替代方案，根据具体操作，该库针对稀疏率在 70%-99.9%之间的矩阵。该库支持多种稀疏矩阵存储格式，包括 COO（坐标格式）、CSR（压缩稀疏行格式）、CSC（压缩稀疏列格式）、BSR（块压缩稀疏行格式）等。主要提供的功能：稀疏向量与稠密向量之间的运算、稠密矩阵与稀疏向量之间的运算、稀疏矩阵与稠密向量之间的运算、稀疏矩阵与稠密矩阵之间的运算、稀疏矩阵与稀疏矩阵之间的运算、稀疏矩阵预处理器等。

2 环境要求

硬件要求：DCU 以及配套驱动，多 DCU 系统；

软件栈要求：DTK（优先使用高版本 DTK）；

运行环境配置：多节点环境要求每个机器的配置完全相同，DCU 可见性，可通过 HIP_VISIBLE_DEVICES 设置。

3 核心概念

3.1 API 使用说明

hipSPARSE 调用流程：

1. 设置矩阵数据布局、批次数和存储格式（例如 CSR、COO 等）。；
2. 设置输入/输出/计算数据类型。允许混合数据类型计算;
3. 设置稀疏向量/矩阵索引的类型（例如 32 位、64 位）；
4. 选择计算算法；
5. 为内部操作确保外部设备内存；
6. 提供跨输入矩阵和向量的广泛一致性检查。包括大小、数据类型、布局、允许的作等的验证；
7. 对输入矩阵和向量进行全面的一致性检查。包括验证大小、数据类型、布局、允许的操作等。

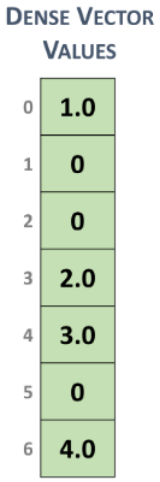
开发者社区

3.2 向量格式

本节介绍密集和稀疏向量格式。

3.2.1 密集向量格式

密集向量由线性存储在内存中的单个数据数组表示，例如以下密集向量（7x1）。

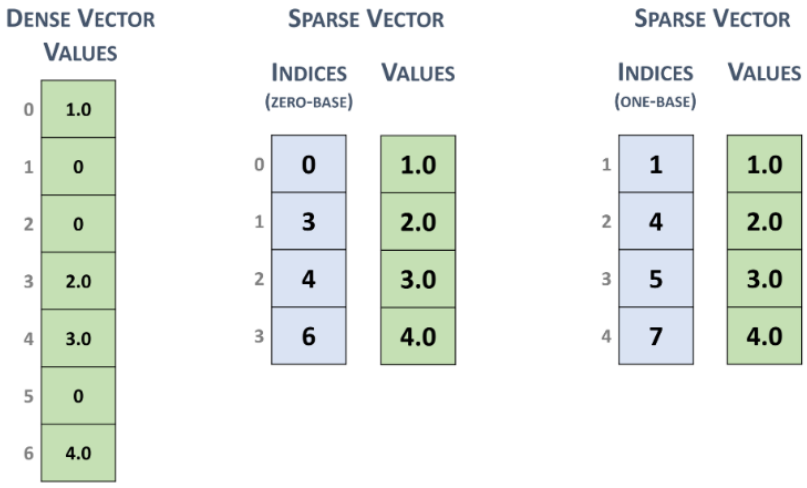


3.2.2 稀疏向量格式

稀疏向量用两个数组表示。

- **values** 数组以密集格式存储等效数组中的非零值。
- **index** 数组以密集格式表示对等数组中相应非零值的位置。

例如，第 3.2.1 节中的密集向量可以存储为具有从零开始或从一开始索引的稀疏向量。



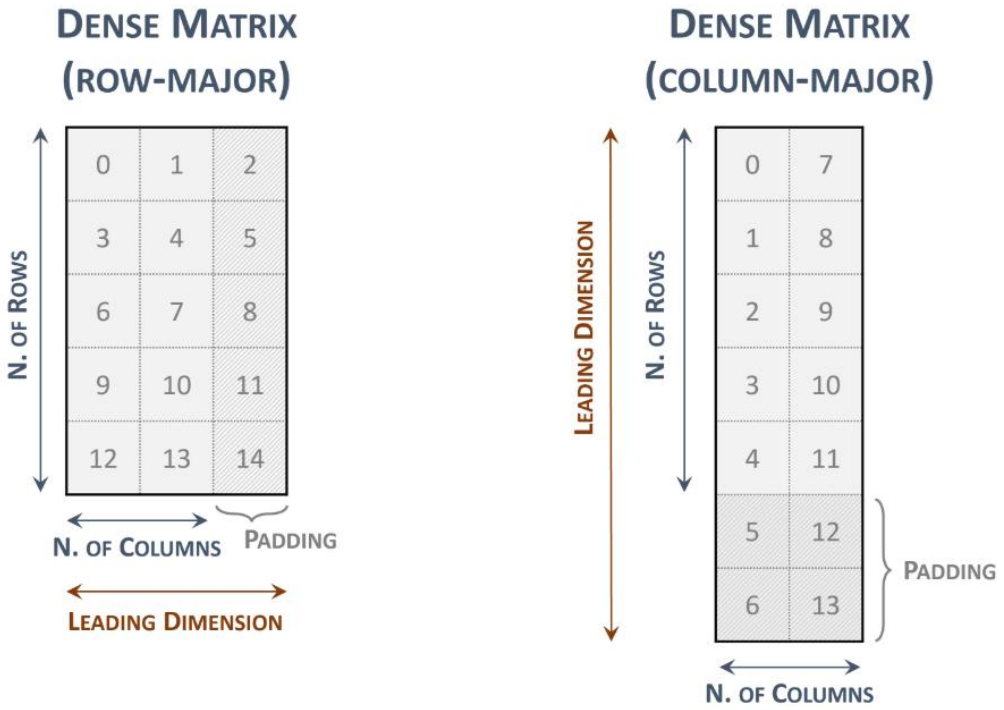
开发者社区

3.3 矩阵格式

本节介绍矩阵的密集存储格式和几种稀疏存储格式。

3.3.1 密集矩阵格式

密集矩阵可以存储在行主和列主内存布局（排序）中，下图表示具有两种内存布局的密集矩阵。

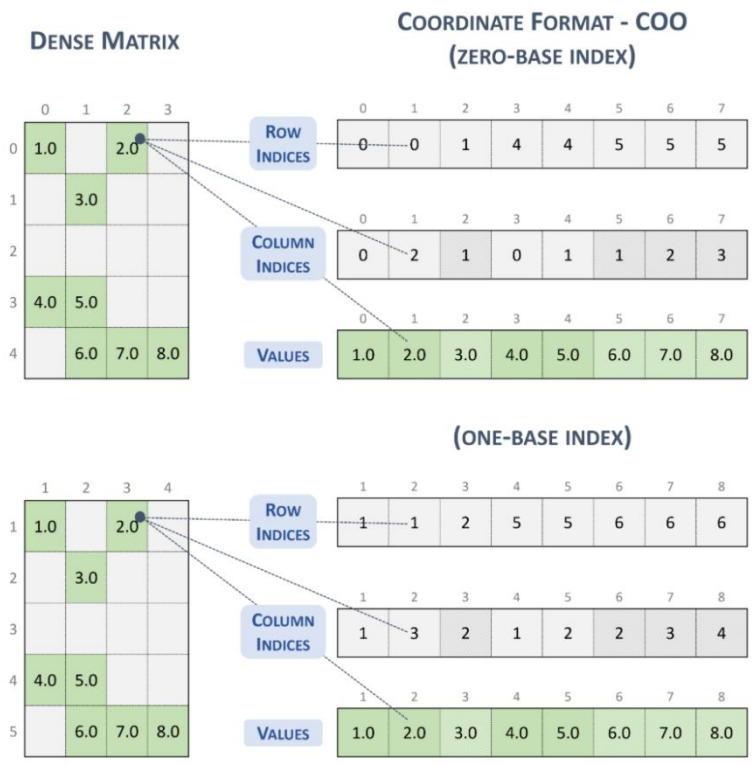


3.3.2 坐标格式（COO）

以 **COO** 格式存储的稀疏矩阵由以下参数表示。

- 矩阵中的**行数 m**。
- 矩阵中的**列数 n**。
- 矩阵中**非零元素的数量 nnz**。
- 指向行索引数组的指针 **Row Indices**，其中包含 *values 数组*中相应元素的行索引。
- 指向列索引数组的指针 **Colum Indices**，其中包含 *values 数组*中相应元素的列索引。
- 指向长度的**值 Values** 数组的指针，该数组以行优先排序保存矩阵的所有非零值。

假定 COO 格式按行排序，以下示例为以 COO 格式表示的 5x4 矩阵。



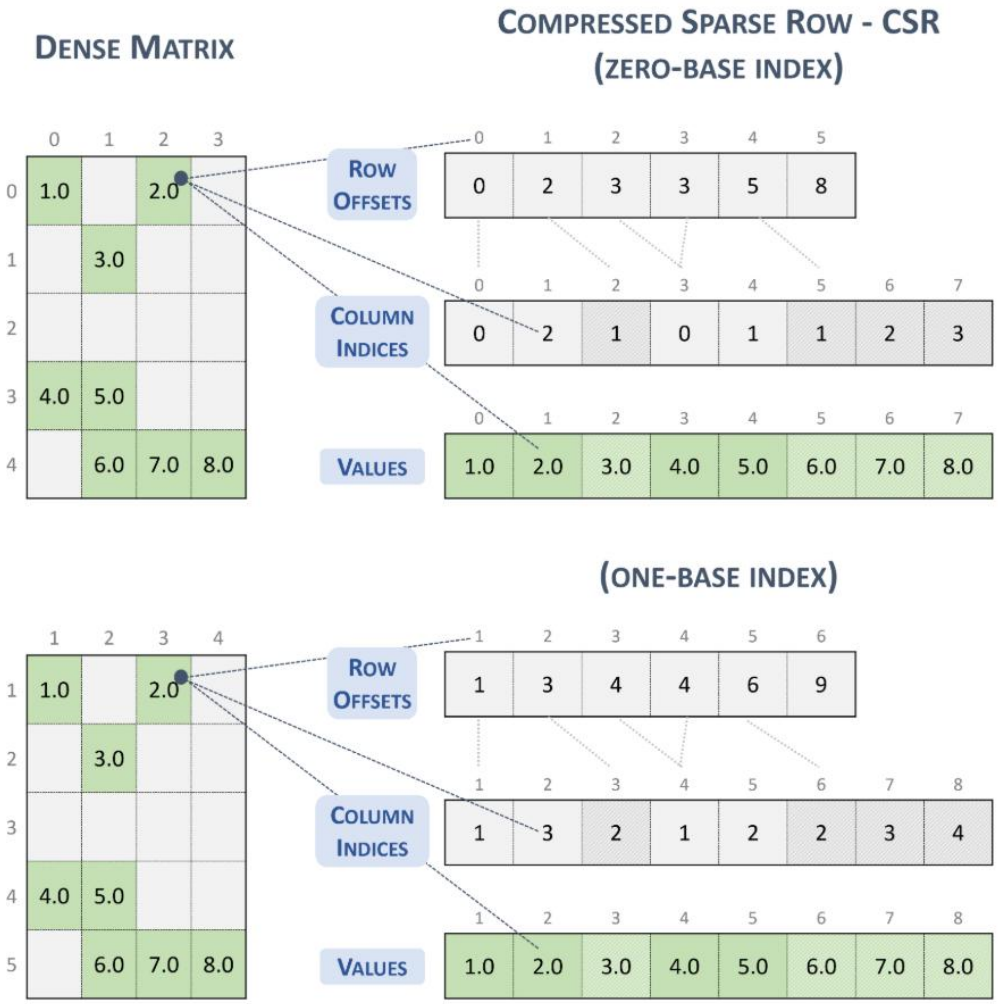
3.3.3 压缩稀疏行格式（CSR）

CSR 格式类似于 COO，其中*行索引*被压缩并替换为*偏移*量数组。

以 CSR 格式存储的稀疏矩阵由以下参数表示。

- 矩阵中的**行数 m**。
- 矩阵中的**列数 n**。
- 矩阵中**非零元素的数量 nnz**。
- 指向行偏移数组的指针 **Row Offsets**，长度为行数 + 1，表示列和值数组中每行的起始位置。
- 指向列索引数组的指针 **Colum Indices**，其中包含 values 数组中相应元素的列索引。
- 指向值数组的指针 **Values**，该数组以行优先排序保存矩阵的所有非零值。

以下示例为以 CSR 格式表示的 5x4 矩阵。

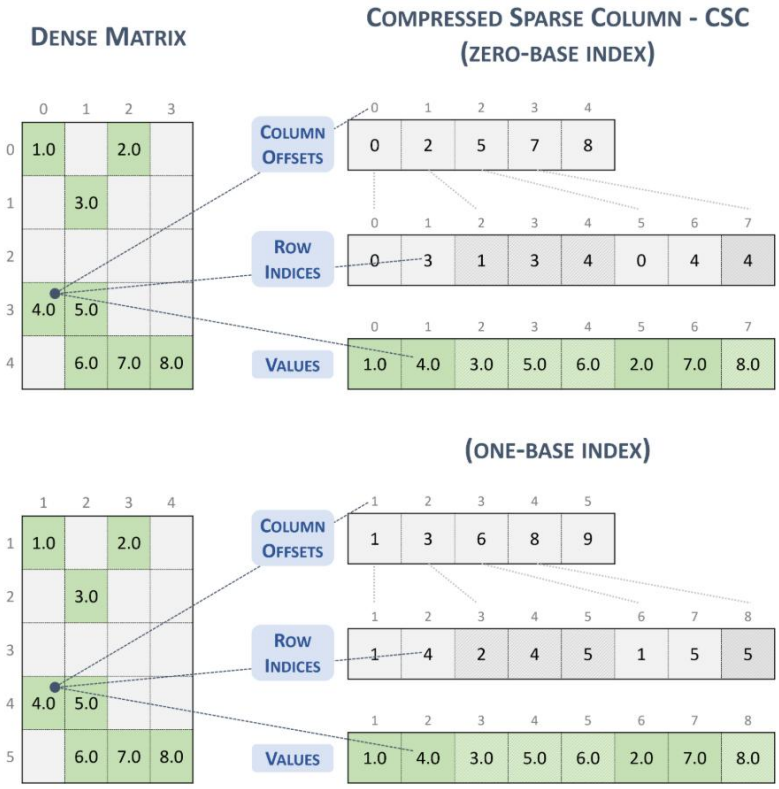


3.3.4 压缩稀疏列格式（CSC）

以 CSC 格式存储的稀疏矩阵由以下参数表示。

- 矩阵中的**行数 m**。
- 矩阵中的**列数 n**。
- 矩阵中**非零元素的数量 nnz**。
- 指向列偏移数组的指针 **Colum Offset**，长度为列数+ 1，表示列和值数组中每列的起始位置。
- 指向行索引数组的指针 **Row Indices**，其中包含 values 数组中相应元素的行索引。
- 指向值数组的指针 **Values**，该数组以列优先排序保存矩阵的所有非零值。

以下示例为以 CSC 格式表示的 5x4 矩阵。



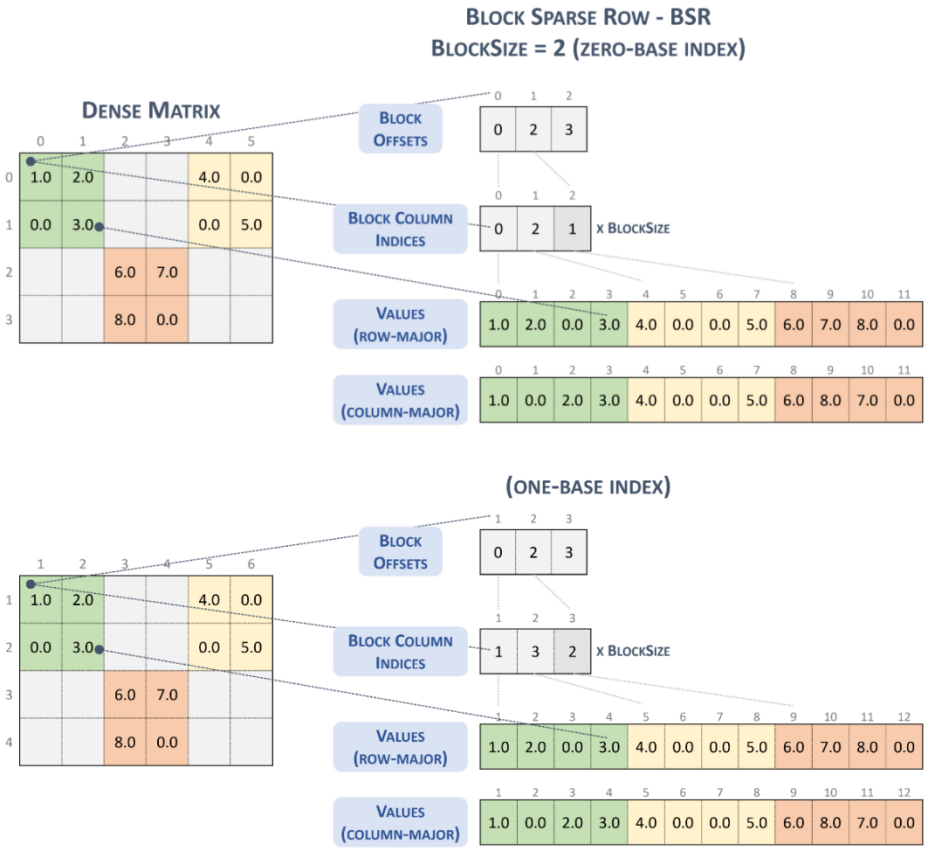
3.3.5 块稀疏行格式（BSR）

BSR 格式类似于 CSR，其中列索引表示二维块而不是单个矩阵数据。块稀疏行格式的矩阵被组织成大小块，由用户定义。BSR 格式以行优先顺序存储块。但是，块的内部存储格式可以是*列优先* 或*行优先* ，与基本索引无关。

以 BSR 格式存储的稀疏矩阵由以下参数表示。

- **块大小 blocksize**。
- 矩阵中的**行块数 mb**。
- 矩阵中的**列块数 nb**。
- 矩阵中**非零块的数量 nnzb**。
- 指向行块偏移数组的指针 **Block Offsets**，行块数 + 1，表示列和值数组中每个行块的起始位置。
- 指向列块索引数组的指针 **Block Column Indices**，其中包含 values 数组中相应元素的位置。
- 指向值数组的指针 **Values**，该数组包含矩阵的所有非零值。

以下示例为以 BSR 格式表示的 4x7 矩阵。



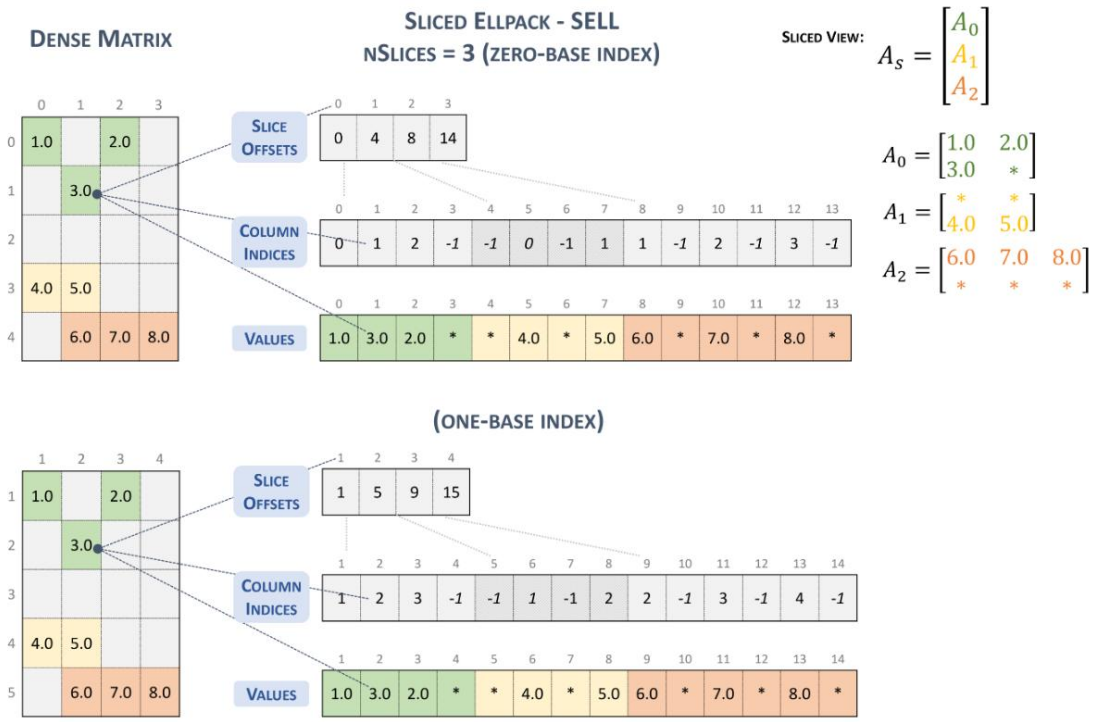
3.3.6 切片稀疏格式（SELL）

SELL 格式的矩阵被划分为由用户定义的精确行数的切片。 为每个切片找到最大行长（即每行的最大非零数），并且切片中的每一行都填充到最大行长。

以 SELL 格式存储的稀疏矩阵由以下参数表示。

- **切片数 NSlices**。
- 矩阵中的**行数 m**。
- 矩阵中的**列数 n**。
- 矩阵中**非零元素的数量 nnz**。
- 指向切片偏移量的指针 **Slice Offsets**，该切片偏移量保存与列和值数组相对应的幻灯片偏移量。
- 指向列索引数组的指针 **Column Indices**，其中包含 values 数组中相应元素的列索引。列索引以列主布局存储。
- 指向值数组的指针 values，该数组保存列主要布局中的所有非零值和填充。

以下示例为 SELL 格式表示的 5x4 矩阵。



3.4 简单 demo

1) api 调用 demo

```
#include <hip/hip_runtime.h>
#include <hipsparse/hipsparse.h>
#include <stdio.h>                    // printf
#include <stdlib.h>                   // EXIT_FAILURE
#include <sys/time.h>

#define dtype float
#define csrIdxType inthipcc singleProcess.cpp -o singleProcess -lrccl
int main(int argc,char** argv)
{
    const int A_num_rows      = 4;
    const int A_num_cols      = 4;
    const int A_nnz           = 9;
    const int nrhs            = 2;
```

```
const int ldb                   = A_num_cols;
const int ldc                   = A_num_rows;
int           hA_csrOffsets[] = { 0, 3, 4, 7, 9 };
int           hA_columns[]      = { 0, 2, 3, 1, 0, 2, 3, 1, 3 };
float         hA_values[]       = { 1.0f, 2.0f, 3.0f, 4.0f, 5.0f,
                                    6.0f, 7.0f, 8.0f, 9.0f };
float         hB[]                = { 1.0f, 8.0f, 23.0f, 52.0f,
                                    1.0f, 8.0f, 23.0f, 52.0f };
float         hC[]                = { 0.0f, 0.0f, 0.0f, 0.0f,
                                    0.0f, 0.0f, 0.0f, 0.0f };
float         hY_result[]       = { 1.0f, 2.0f, 3.0f, 4.0f,
                                    1.0f, 2.0f, 3.0f, 4.0f };
float         alpha               = 1.0f;
// Device memory management
int     *dA_csrOffsets, *dA_columns;
float *dA_values, *dB, *dC;
hipMalloc((void**) &dA_csrOffsets,
                                (A_num_rows + 1) * sizeof(int));
hipMalloc((void**) &dA_columns, A_nnz * sizeof(int));
hipMalloc((void**) &dA_values,    A_nnz * sizeof(float));
hipMalloc((void**) &dB,             nrhs * A_num_cols * sizeof(float));
hipMalloc((void**) &dC,             nrhs * A_num_rows * sizeof(float));

hipMemcpy(dA_csrOffsets, hA_csrOffsets,
                                (A_num_rows + 1) * sizeof(int),
                                hipMemcpyHostToDevice) ;
hipMemcpy(dA_columns, hA_columns, A_nnz * sizeof(int),
                                hipMemcpyHostToDevice) ;
hipMemcpy(dA_values, hA_values, A_nnz * sizeof(float),
```

```
                                    hipMemcpyHostToDevice) ;
  hipMemcpy(dB, hB, nrhs * A_num_cols * sizeof(float),
                                    hipMemcpyHostToDevice) ;
  hipMemcpy(dC, hC, nrhs * A_num_rows * sizeof(float),
                                    hipMemcpyHostToDevice) ;
 // HIPSPARSE APIs
  hipsparseHandle_t          handle = NULL;
  hipsparseSpMatDescr_t matA;
  hipsparseDnMatDescr_t matB, matC;
  void*                       dBuffer       = NULL;
  size_t                      bufferSize = 0;
  hipsparseSpSMDescr_t    spsmDescr;
  hipsparseCreate(&handle);
  // Create sparse matrix A in CSR format
  hipsparseCreateCsr(&matA, A_num_rows, A_num_cols, A_nnz,
                          dA_csrOffsets, dA_columns, dA_values,
                  HIPSPARSE_INDEX_32I, HIPSPARSE_INDEX_32I,
                  HIPSPARSE_INDEX_BASE_ZERO, HIP_R_32F);
  // Create dense matrix B
  hipsparseCreateDnMat(&matB, A_num_cols, nrhs, ldb, dB,
                   HIP_R_32F, HIPSPARSE_ORDER_COL) ;
  // Create dense matrix C
  hipsparseCreateDnMat(&matC, A_num_rows, nrhs, ldc, dC,
                HIP_R_32F, HIPSPARSE_ORDER_COL) ;
    // Create opaque data structure, that holds analysis data between calls.
  hipsparseSpSM_createDescr(&spsmDescr);
  // Specify Lower|Upper fill mode.
  hipsparseFillMode_t fillmode = HIPSPARSE_FILL_MODE_LOWER;
  hipsparseSpMatSetAttribute(matA,    HIPSPARSE_SPMAT_FILL_MODE,
```

```
        &fillmode, sizeof(fillmode));
    // Specify Unit|Non-Unit diagonal type.
    hipsparseDiagType_t diagtype = HIPSPARSE_DIAG_TYPE_NON_UNIT;
    hipsparseSpMatSetAttribute(matA, HIPSPARSE_SPMAT_DIAG_TYPE,
                                    &diagtype, sizeof(diagtype));
    // allocate an external buffer for analysis
    hipsparseSpSM_bufferSize(handle,
    HIPSPARSE_OPERATION_NON_TRANSPOSE,
      HIPSPARSE_OPERATION_NON_TRANSPOSE,
      &alpha, matA, matB, matC,    HIP_R_32F,
      HIPSPARSE_SPSM_ALG_DEFAULT, spsmDescr,&bufferSize) ;
    hipMalloc(&dBuffer, bufferSize) ;
       hipsparseSpSM_analysis(handle,
HIPSPARSE_OPERATION_NON_TRANSPOSE,HIPSPARSE_OPERATION_NON
_TRANSPOSE,&alpha,          matA,          matB,          matC,
HIP_R_32F,HIPSPARSE_SPSM_ALG_DEFAULT, spsmDescr, dBuffer) ;
    // exe hipte SpSM
       hipsparseSpSM_solve(handle,
HIPSPARSE_OPERATION_NON_TRANSPOSE,
HIPSPARSE_OPERATION_NON_TRANSPOSE
    &alpha, matA, matB, matC,    HIP_R_32F,
      HIPSPARSE_SPSM_ALG_DEFAULT, spsmDescr) ;
    hipMemcpy(hC, dC, nrhs * A_num_rows * sizeof(float),
                                    hipMemcpyDeviceToHost);
    int j = 0;
    int correct = 1;
    for (int i = 0; i < nrhs * A_num_rows; i++) {
          if (hC[i] != hY_result[i]) { // direct floating point comparison is not
                correct = 0;                    // reliable
```

```
                break;
            }
        }
        if (correct)
            printf("spsm_csr_example test PASSED\n");
        else
            printf("spsm_csr_example test FAILED: wrong result\n");
        hipsparseDestroySpMat(matA) ;
        hipsparseDestroyDnMat(matB) ;
        hipsparseDestroyDnMat(matC) ;
        hipsparseSpSM_destroyDescr(spsmDescr);
        hipsparseDestroy(handle) ;
        hipFree(dBuffer);
        hipFree(dA_csrOffsets);
        hipFree(dA_columns) ;
        hipFree(dA_values) ;
        hipFree(dB) ;
        hipFree(dC) ;
        // A_num_rows -= 2000;
        // A_num_cols -= 2000;
        }
        return EXIT_SUCCESS;
}
```

```
hipcc spsm-csr-my-roc.cpp -o spsm-csr -lhipsparse
./spsm-csr-my-roc
```

4 API 详解

4.1 hipSPARSE 辅助函数

4.1.1 hipsparseCreate

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseCreate(hipsparseHandle_t* handle);
/*! \ingroup aux_module
\brief Create a hipsparse handle
\details
\p hipsparseCreate creates the hipSPARSE library context. It must be
initialized before any other hipSPARSE API function is invoked and must be passed to
all subsequent library function calls. The handle should be destroyed at the end
using hipsparseDestroy().
*/
```

4.1.2 hipsparseDestroy

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseDestroy(hipsparseHandle_t handle);
/*! \ingroup aux_module
\brief Destroy a hipsparse handle

\details
\p hipsparseDestroy destroys the hipSPARSE library context and releases all
resources used by the hipSPARSE library.
*/
```

4.1.3 hipsparseGetErrorName

```
HIPSPARSE_EXPORT
const char* hipsparseGetErrorName(hipsparseStatus_t status);
/*! \ingroup aux_module
```

```
\brief Return the string representation of a hipSPARSE status's matching backend status enum name
\details
\p hipsparseGetErrorName takes a hipSPARSE status as input and first converts it to the matching backend
status (either rocsparse_status or cusparseStatus_t). It then returns the string representation of this status
enum name. If the status is not recognized, the function returns "Unrecognized status code".
For example, hipsparseGetErrorName(HIPSPARSE_STATUS_SUCCESS) on a system with a rocSPARSE backend will
return "rocsparse_status_success". On a system with a cuSPARSE backend this function would return
"CUSPARSE_STATUS_SUCCESS".
*/
```

4.1.4 hipsparseGetErrorString

```
HIPSPARSE_EXPORT
const char* hipsparseGetErrorString(hipsparseStatus_t status);
/*! \ingroup aux_module
\brief Return the hipSPARSE status's matching backend status description as a string
\details
\p hipsparseGetErrorString takes a hipSPARSE status as input and first converts it to the matching backend
status (either rocsparse_status or cusparseStatus_t). It then returns the string description of this status.
If the status is not recognized, the function returns "Unrecognized status code".
*/
```

4.1.5 hipsparseGetVersion

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseGetVersion(hipsparseHandle_t handle, int* version);
/*! \ingroup aux_module
\brief Get hipSPARSE version
\details
\p hipsparseGetVersion gets the hipSPARSE library version number.
patch = version % 100
minor = version / 100 % 1000
major = version / 100000
*/
```

4.1.6 hipsparseGetProperty

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseGetProperty(hipLibraryPropertyType type, int* value);
/**
\brief Get hipSPARSE library property
*/
hipsparseGetGitRevision
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseGetGitRevision(hipsparseHandle_t handle, char* rev);
/*! \ingroup aux_module
\brief Get hipSPARSE git revision
\details
\p hipsparseGetGitRevision gets the hipSPARSE library git commit revision (SHA-1).
*/
```

4.1.7 hipsparseSetStream

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseSetStream(hipsparseHandle_t handle, hipStream_t
```

```
streamId);
/*! \ingroup aux_module
\brief Specify user defined HIP stream
\details
\p hipsparseSetStream specifies the stream to be used by the hipSPARSE library
context and all subsequent function calls.
*/
```

4.1.8 hipsparseGetStream

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseGetStream(hipsparseHandle_t handle, hipStream_t*
streamId);
/*! \ingroup aux_module
\brief Get current stream from library context
\details
\p hipsparseGetStream gets the hipSPARSE library context stream which is currently
used for all subsequent function calls.
*/
```

4.1.9 hipsparseSetPointerMode

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseSetPointerMode(hipsparseHandle_t handle,
hipsparsePointerMode_t mode);
/*! \ingroup aux_module
\brief Specify pointer mode
\details
\p hipsparseSetPointerMode specifies the pointer mode to be used by the hipSPARSE
library context and all subsequent function calls. By default, all values are passed
by reference on the host. Valid pointer modes are \ref
HIPSPARSE_POINTER_MODE_HOST
```

```
or \ref HIPSPARSE_POINTER_MODE_DEVICE.
*/
```

4.1.10 hipsparseGetPointerMode

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseGetPointerMode(hipsparseHandle_t handle,
hipsparsePointerMode_t* mode);
/*! \ingroup aux_module
\brief Get current pointer mode from library context
\details
\p hipsparseGetPointerMode gets the hipSPARSE library context pointer mode which
is currently used for all subsequent function calls.
*/
```

4.1.11 hipsparseCreateMatDescr

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseCreateMatDescr(hipsparseMatDescr_t* descrA);
/*! \ingroup aux_module
\brief Create a matrix descriptor
\details
\p hipsparseCreateMatDescr creates a matrix descriptor. It initializes
\ref hipsparseMatrixType_t to \ref HIPSPARSE_MATRIX_TYPE_GENERAL and
\ref hipsparseIndexBase_t to \ref HIPSPARSE_INDEX_BASE_ZERO. It should be
destroyed
at the end using hipsparseDestroyMatDescr().
*/
```

4.1.12 hipsparseDestroyMatDescr

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseDestroyMatDescr(hipsparseMatDescr_t descrA);
/*! \ingroup aux_module
```

```
\brief Destroy a matrix descriptor
\details
\p hipsparseDestroyMatDescr destroys a matrix descriptor and releases all
resources used by the descriptor.
*/
```

4.1.13 hipsparseCopyMatDescr

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseCopyMatDescr(hipsparseMatDescr_t dest, const
hipsparseMatDescr_t src);
/*! \ingroup aux_module
\brief Copy a matrix descriptor
\details
\p hipsparseCopyMatDescr copies a matrix descriptor. Both, source and destination
matrix descriptors must be initialized prior to calling \p hipsparseCopyMatDescr.
*/
```

4.1.14 hipsparseSetMatType

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseSetMatType(hipsparseMatDescr_t descrA,
hipsparseMatrixType_t type);
/*! \ingroup aux_module
\brief Specify the matrix type of a matrix descriptor
\details
\p hipsparseSetMatType sets the matrix type of a matrix descriptor. Valid
matrix types are \ref HIPSPARSE_MATRIX_TYPE_GENERAL,
\ref HIPSPARSE_MATRIX_TYPE_SYMMETRIC, \ref
HIPSPARSE_MATRIX_TYPE_HERMITIAN or
\ref HIPSPARSE_MATRIX_TYPE_TRIANGULAR.
*/
```

4.1.15 hipsparseGetMatType

```
HIPSPARSE_EXPORT
hipsparseMatrixType_t hipsparseGetMatType(const hipsparseMatDescr_t descrA);
/*! \ingroup aux_module
\brief Get the matrix type of a matrix descriptor
\details
\p hipsparseGetMatType returns the matrix type of a matrix descriptor.
*/
```

4.1.16 hipsparseSetMatFillMode

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseSetMatFillMode(hipsparseMatDescr_t descrA,
hipsparseFillMode_t fillMode);
/*! \ingroup aux_module
\brief Specify the matrix fill mode of a matrix descriptor
\details
\p hipsparseSetMatFillMode sets the matrix fill mode of a matrix descriptor.
Valid fill modes are \ref HIPSPARSE_FILL_MODE_LOWER or
\ref HIPSPARSE_FILL_MODE_UPPER.
*/
```

4.1.17 hipsparseGetMatFillMode

```
HIPSPARSE_EXPORT
hipsparseFillMode_t hipsparseGetMatFillMode(const hipsparseMatDescr_t descrA);
/*! \ingroup aux_module
\brief Get the matrix fill mode of a matrix descriptor
\details
\p hipsparseGetMatFillMode returns the matrix fill mode of a matrix descriptor.
*/
```

4.1.18 hipsparseSetMatDiagType

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseSetMatDiagType(hipsparseMatDescr_t descrA,
hipsparseDiagType_t diagType);
/*! \ingroup aux_module
\brief Specify the matrix diagonal type of a matrix descriptor
\details
\p hipsparseSetMatDiagType sets the matrix diagonal type of a matrix
descriptor. Valid diagonal types are \ref HIPSPARSE_DIAG_TYPE_UNIT or
\ref HIPSPARSE_DIAG_TYPE_NON_UNIT.
*/
```

4.1.19 hipsparseGetMatDiagType

```
HIPSPARSE_EXPRT
hipsparseDiagType_t hipsparseGetMatDiagType(const hipsparseMatDescr_t descrA);
/*! \ingroup aux_modul
\brief Get the matrix diagonal type of a matrix descriptor
\details
\p hipsparseGetMatDiagType returns the matrix diagonal type of a matrix
descriptor.
*/
```

4.1.20 hipsparseSetMatIndexBase

```
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseSetMatIndexBase(hipsparseMatDescr_t descrA,
hipsparseIndexBase_t base);
/*! \ingroup aux_module
\brief Specify the index base of a matrix descriptor
\details
\p hipsparseSetMatIndexBase sets the index base of a matrix descriptor. Valid
```

```
options are \ref HIPSPARSE_INDEX_BASE_ZERO or \ref
HIPSPARSE_INDEX_BASE_ONE.
*/
```

4.1.21 hipsparseGetMatIndexBase

```
HIPSPARSE_EXPORT
hipsparseIndexBase_t hipsparseGetMatIndexBase(const hipsparseMatDescr_t descrA);
/*! \ingroup aux_module
\brief Get the index base of a matrix descriptor
\details
\p hipsparseGetMatIndexBase returns the index base of a matrix descriptor.
*/
```

4.1.22 hipsparseCreateHybMat

```
DEPRECATED_CUDA_10000("The routine will be removed in CUDA 11")
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseCreateHybMat(hipsparseHybMat_t* hybA);
/*! \ingroup aux_module
\brief Create a \p HYB matrix structure
\details
\p hipsparseCreateHybMat creates a structure that holds the matrix in \p HYB
storage format. It should be destroyed at the end using hipsparseDestroyHybMat().
*/
```

4.1.23 hipsparseDestroyHybMat

```
DEPRECATED_CUDA_10000("The routine will be removed in CUDA 11")
HIPSPARSE_EXPORT
hipsparseStatus_t hipsparseDestroyHybMat(hipsparseHybMat_t hybA);
/*! \ingroup aux_module
\brief Destroy a \p HYB matrix structure
\details
```

```
\p hipsparseDestroyHybMat destroys a \p HYB structure.
*/
ult_code for more details
        @param[out] uniqueId        Pointer to where uniqueId will be stored */
```

4.2 level 1 函数

4.2.1 hipsparseXaxpyi

```
hipsparseStatus_t hipsparseSaxpyi(hipsparseHandle_t handle, int nnz, const float *alpha, const float *xVal, const int *xInd, float *y, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseDaxpyi(hipsparseHandle_t handle, int nnz, const double *alpha, const double *xVal, const int *xInd, double *y, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseCaxpyi(hipsparseHandle_t handle, int nnz, const hipComplex *alpha, const hipComplex *xVal, const int *xInd, hipComplex *y, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseZaxpyi(hipsparseHandle_t handle, int nnz, const hipDoubleComplex *alpha, const hipDoubleComplex *xVal, const int *xInd, hipDoubleComplex *y, hipsparseIndexBase_t idxBase)
/*
Scale a sparse vector and add it to a dense vector.
hipsparseXaxpyi multiplies the sparse vector    with scalar    and adds the result to the dense vector , such that
for(i = 0; i < nnz; ++i)
{
    y[x_ind[i]] = y[x_ind[i]] + alpha * x_val[i];
}
*/
```

4.2.2 hipsparseXdoti

```
hipsparseStatus_t hipsparseSdoti(hipsparseHandle_t handle, int nnz, const float *xVal, const int *xInd, const float *y, float *result, hipsparseIndexBase_t idxBase)
```

```
hipsparseStatus_t hipsparseDdoti(hipsparseHandle_t handle, int nnz, const double *xVal, const int *xInd, const double *y, double *result, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseCdoti(hipsparseHandle_t handle, int nnz, const hipComplex *xVal, const int *xInd, const hipComplex *y, hipComplex *result, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseZdoti(hipsparseHandle_t handle, int nnz, const hipDoubleComplex *xVal, const int *xInd, const hipDoubleComplex *y, hipDoubleComplex *result, hipsparseIndexBase_t idxBase)
/*
Compute the dot product of a sparse vector with a dense vector.
hipsparseXdoti computes the dot product of the sparse vector with the dense vector
for(i = 0; i < nnz; ++i)
{
    result += x_val[i] * y[x_ind[i]];
}
*/
```

4.2.3 hipsparseXdotci

```
hipsparseStatus_t hipsparseCdotci(hipsparseHandle_t handle, int nnz, const hipComplex *xVal, const int *xInd, const hipComplex *y, hipComplex *result, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseZdotci(hipsparseHandle_t handle, int nnz, const hipDoubleComplex *xVal, const int *xInd, const hipDoubleComplex *y, hipDoubleComplex *result, hipsparseIndexBase_t idxBase)
/*
Compute the dot product of a complex conjugate sparse vector with a dense vector.
hipsparseXdotci computes the dot product of the complex conjugate sparse vector with the dense vector.
for(i = 0; i < nnz; ++i){
    result += conj(x_val[i]) * y[x_ind[i]];
```

```
}
*/
```

4.2.4 hipsparseXgthr

```
hipsparseStatus_t hipsparseSgthr(hipsparseHandle_t handle, int nnz, const float *y, float *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseDgthr(hipsparseHandle_t handle, int nnz, const double *y, double *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseCgthr(hipsparseHandle_t handle, int nnz, const hipComplex *y, hipComplex *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseZgthr(hipsparseHandle_t handle, int nnz, const hipDoubleComplex *y, hipDoubleComplex *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
/*
Gather elements from a dense vector and store them into a sparse vector.
hipsparseXgthr gathers the elements that are listed in x_ind from the dense vector
  and stores them in the sparse vector
for(i = 0; i < nnz; ++i)
{
      x_val[i] = y[x_ind[i]];
}
*/
```

4.2.5 hipsparseXgthrz

```
hipsparseStatus_t hipsparseSgthrz(hipsparseHandle_t handle, int nnz, float *y, float *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseDgthrz(hipsparseHandle_t handle, int nnz, double *y, double *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseCgthrz(hipsparseHandle_t handle, int nnz, hipComplex *y, hipComplex *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
```

```
hipsparseStatus_t hipsparseZgthrz(hipsparseHandle_t handle, int nnz, hipDoubleComplex *y, hipDoubleComplex *xVal, const int *xInd, hipsparseIndexBase_t idxBase)
/*
Gather and zero out elements from a dense vector and store them into a sparse vector.
hipsparseXgthrz gathers the elements that are listed in x_ind from the dense vector and stores them in the sparse vector. The gathered elements are replaced by zero.
for(i = 0; i < nnz; ++i)
{
    x_val[i]    = y[x_ind[i]];
    y[x_ind[i]] = 0;
}
*/
```

4.2.6 hipsparseXroti()

```
hipsparseStatus_t hipsparseSroti(hipsparseHandle_t handle, int nnz, float *xVal, const int *xInd, float *y, const float *c, const float *s, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseDroti(hipsparseHandle_t handle, int nnz, double *xVal, const int *xInd, double *y, const double *c, const double *s, hipsparseIndexBase_t idxBase)
/*
Apply Givens rotation to a dense and a sparse vector.
hipsparseXroti applies the Givens rotation matrix to the sparse vector and the dense vector.
for(i = 0; i < nnz; ++i)
{
    x_tmp = x_val[i];
    y_tmp = y[x_ind[i]];
    x_val[i]    = c * x_tmp + s * y_tmp;
    y[x_ind[i]] = c * y_tmp - s * x_tmp;
```

```
}
*/
```

4.2.7 hipsparseXsctr()

```
hipsparseStatus_t hipsparseSsctr(hipsparseHandle_t handle, int nnz, const float *xVal, const int *xInd, float *y, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseDsctr(hipsparseHandle_t handle, int nnz, const double *xVal, const int *xInd, double *y, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseCsctr(hipsparseHandle_t handle, int nnz, const hipComplex *xVal, const int *xInd, hipComplex *y, hipsparseIndexBase_t idxBase)
hipsparseStatus_t hipsparseZsctr(hipsparseHandle_t handle, int nnz, const hipDoubleComplex *xVal, const int *xInd, hipDoubleComplex *y, hipsparseIndexBase_t idxBase)
/*
Scatter elements from a dense vector across a sparse vector.
hipsparseXsctr scatters the elements that are listed in x_ind from the sparse vector into the dense vector . Indices of  that are not listed in x_ind remain unchanged.
for(i = 0; i < nnz; ++i)
{
    y[x_ind[i]] = x_val[i];
}
*/
```

4.3 level 2 函数

4.3.1 hipsparseXcsrmv()

```
hipsparseStatus_t hipsparseScsrmv(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, int nnz, const float *alpha, const hipsparseMatDescr_t descrA, const float *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const float *x, const float *beta, float *y)
hipsparseStatus_t hipsparseDcsrmv(hipsparseHandle_t handle, hipsparseOperation_t
```

```
transA, int m, int n, int nnz, const double *alpha, const hipsparseMatDescr_t descrA, const double *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const double *x, const double *beta, double *y)
hipsparseStatus_t hipsparseCcsrmv(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, int nnz, const hipComplex *alpha, const hipsparseMatDescr_t descrA, const hipComplex *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const hipComplex *x, const hipComplex *beta, hipComplex *y)
hipsparseStatus_t hipsparseZcsrmv(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, int nnz, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descrA, const hipDoubleComplex *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const hipDoubleComplex *x, const hipDoubleComplex *beta, hipDoubleComplex *y)
/*
for(i = 0; i < m; ++i)
{
    y[i] = beta * y[i];
    for(j = csr_row_ptr[i]; j < csr_row_ptr[i + 1]; ++j)
    {
        y[i] = y[i] + alpha * csr_val[j] * x[csr_col_ind[j]];
    }
}
*/
```

4.3.2 hipsparseXhybmv

```
hipsparseStatus_t hipsparseShybmv(hipsparseHandle_t handle, hipsparseOperation_t transA, const float *alpha, const hipsparseMatDescr_t descrA, const hipsparseHybMat_t hybA, const float *x, const float *beta, float *y)
hipsparseStatus_t hipsparseDhybmv(hipsparseHandle_t handle, hipsparseOperation_t transA, const double *alpha, const hipsparseMatDescr_t descrA, const hipsparseHybMat_t hybA, const double *x, const double *beta, double *y)
```

```
hipsparseStatus_t hipsparseChybmv(hipsparseHandle_t handle, hipsparseOperation_t transA, const hipComplex *alpha, const hipsparseMatDescr_t descrA, const hipsparseHybMat_t hybA, const hipComplex *x, const hipComplex *beta, hipComplex *y)
hipsparseStatus_t hipsparseZhybmv(hipsparseHandle_t handle, hipsparseOperation_t transA, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descrA, const hipsparseHybMat_t hybA, const hipDoubleComplex *x, const hipDoubleComplex *beta, hipDoubleComplex *y)
/*
Sparse matrix vector multiplication using HYB storage format.
hipsparseXhybmv multiplies the scalar with a sparse matrix, defined in HYB storage format, and the dense vector and adds the result to the dense vector that is multiplied by the scalar.
*/
```

4.3.3 hipsparseXbsrmv

```
hipsparseStatus_t hipsparseSbsrmv(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nb, int nnzb, const float *alpha, const hipsparseMatDescr_t descrA, const float *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, const float *x, const float *beta, float *y)
hipsparseStatus_t hipsparseDbsrmv(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nb, int nnzb, const double *alpha, const hipsparseMatDescr_t descrA, const double *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, const double *x, const double *beta, double *y)
hipsparseStatus_t hipsparseCbsrmv(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nb, int nnzb, const hipComplex *alpha, const hipsparseMatDescr_t descrA, const hipComplex *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, const hipComplex *x,
```

const hipComplex *beta, hipComplex *y)

hipsparseStatus_t hipsparseZbsrmv(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nb, int nnzb, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descrA, const hipDoubleComplex *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, const hipDoubleComplex *x, const hipDoubleComplex *beta, hipDoubleComplex *y)

/*

Sparse matrix vector multiplication using BSR storage format.

hipsparseXbsrmv multiplies the scalar with a sparse matrix, defined in BSR storage format, and the dense vector and adds the result to the dense vector that is multiplied by the scalar.

*/

4.3.4 hipsparseXbsrxmv()

hipsparseStatus_t hipsparseSbsrxmv(hipsparseHandle_t handle, hipsparseDirection_t dir, hipsparseOperation_t trans, int sizeOfMask, int mb, int nb, int nnzb, const float *alpha, const hipsparseMatDescr_t descr, const float *bsrVal, const int *bsrMaskPtr, const int *bsrRowPtr, const int *bsrEndPtr, const int *bsrColInd, int blockDim, const float *x, const float *beta, float *y)

hipsparseStatus_t hipsparseDbsrxmv(hipsparseHandle_t handle, hipsparseDirection_t dir, hipsparseOperation_t trans, int sizeOfMask, int mb, int nb, int nnzb, const double *alpha, const hipsparseMatDescr_t descr, const double *bsrVal, const int *bsrMaskPtr, const int *bsrRowPtr, const int *bsrEndPtr, const int *bsrColInd, int blockDim, const double *x, const double *beta, double *y)

hipsparseStatus_t hipsparseCbsrxmv(hipsparseHandle_t handle, hipsparseDirection_t dir, hipsparseOperation_t trans, int sizeOfMask, int mb, int nb, int nnzb, const hipComplex *alpha, const hipsparseMatDescr_t descr, const hipComplex *bsrVal, const int *bsrMaskPtr, const int *bsrRowPtr, const int *bsrEndPtr, const int *bsrColInd, int blockDim, const hipComplex *x, const hipComplex *beta, hipComplex *y)

hipsparseStatus_t hipsparseZbsrxmv(hipsparseHandle_t handle, hipsparseDirection_t

dir, hipsparseOperation_t trans, int sizeOfMask, int mb, int nb, int nnzb, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descr, const hipDoubleComplex *bsrVal, const int *bsrMaskPtr, const int *bsrRowPtr, const int *bsrEndPtr, const int *bsrColInd, int blockDim, const hipDoubleComplex *x, const hipDoubleComplex *beta, hipDoubleComplex *y)

/*

Sparse matrix vector multiplication with mask operation using BSR storage format.

hipsparseXbsrxmv multiplies the scalar with a sparse modified matrix, defined in BSR storage format, and the dense vector and adds the result to the dense vector that is multiplied by the scalar.

*/

4.3.5 hipsparseXbsrsv2_solve

hipsparseStatus_t hipsparseSbsrsv2_solve(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nnzb, const float *alpha, const hipsparseMatDescr_t descrA, const float *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrsv2Info_t info, const float *f, float *x, hipsparseSolvePolicy_t policy, void *pBuffer)

hipsparseStatus_t hipsparseDbsrsv2_solve(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nnzb, const double *alpha, const hipsparseMatDescr_t descrA, const double *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrsv2Info_t info, const double *f, double *x, hipsparseSolvePolicy_t policy, void *pBuffer)

hipsparseStatus_t hipsparseCbsrsv2_solve(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nnzb, const hipComplex *alpha, const hipsparseMatDescr_t descrA, const hipComplex *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrsv2Info_t info, const hipComplex *f, hipComplex *x, hipsparseSolvePolicy_t policy, void *pBuffer)

hipsparseStatus_t hipsparseZbsrsv2_solve(hipsparseHandle_t handle,

hipsparseDirection_t dirA, hipsparseOperation_t transA, int mb, int nnzb, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descrA, const hipDoubleComplex *bsrSortedValA, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrsv2Info_t info, const hipDoubleComplex *f, hipDoubleComplex *x, hipsparseSolvePolicy_t policy, void *pBuffer)

/*

Sparse triangular solve using BSR storage format.

hipsparseXbsrsv2_solve solves a sparse triangular linear system of a sparse matrix, defined in BSR storage format, a dense solution vector and the right-hand side that is multiplied.

hipsparseXbsrsv2_solve requires a user allocated temporary buffer. Its size is returned by hipsparseXbsrsv2_bufferSize() or hipsparseXbsrsv2_bufferSizeExt(). Furthermore, analysis meta data is required. It can be obtained by hipsparseXbsrsv2_analysis(). hipsparseXbsrsv2_solve reports the first zero pivot (either numerical or structural zero). The zero pivot status can be checked calling hipsparseXbsrsv2_zeroPivot(). If hipsparseDiagType_t == HIPSPARSE_DIAG_TYPE_UNIT, no zero pivot will be reported.

*/

4.3.6 hipsparseXgemvi

hipsparseStatus_t hipsparseSgemvi(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, const float *alpha, const float *A, int lda, int nnz, const float *x, const int *xInd, const float *beta, float *y, hipsparseIndexBase_t idxBase, void *pBuffer)

hipsparseStatus_t hipsparseDgemvi(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, const double *alpha, const double *A, int lda, int nnz, const double *x, const int *xInd, const double *beta, double *y, hipsparseIndexBase_t idxBase, void *pBuffer)

hipsparseStatus_t hipsparseCgemvi(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, const hipComplex *alpha, const hipComplex *A, int lda, int nnz,

const hipComplex *x, const int *xInd, const hipComplex *beta, hipComplex *y, hipsparseIndexBase_t idxBase, void *pBuffer)

hipsparseStatus_t hipsparseZgemvi(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, const hipDoubleComplex *alpha, const hipDoubleComplex *A, int lda, int nnz, const hipDoubleComplex *x, const int *xInd, const hipDoubleComplex *beta, hipDoubleComplex *y, hipsparseIndexBase_t idxBase, void *pBuffer)

/*

Dense matrix sparse vector multiplication.

hipsparseXgemvi multiplies the scalar with a dense matrix and the sparse vector and adds the result to the dense vector that is multiplied by the scalar.

hipsparseXgemvi requires a user allocated temporary buffer. Its size is returned by hipsparseXgemvi_bufferSize().

*/

4.4 Level 3 函数

4.4.1 hipsparseXbsrmm

hipsparseStatus_t hipsparseSbsrmm(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, hipsparseOperation_t transB, int mb, int n, int kb, int nnzb, const float *alpha, const hipsparseMatDescr_t descrA, const float *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, const float *B, int ldb, const float *beta, float *C, int ldc)

hipsparseStatus_t hipsparseDbsrmm(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, hipsparseOperation_t transB, int mb, int n, int kb, int nnzb, const double *alpha, const hipsparseMatDescr_t descrA, const double *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, const double *B, int ldb, const double *beta, double *C, int ldc)

hipsparseStatus_t hipsparseCbsrmm(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, hipsparseOperation_t transB, int mb, int n, int kb, int nnzb, const hipComplex *alpha, const hipsparseMatDescr_t descrA, const

hipComplex *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, const hipComplex *B, int ldb, const hipComplex *beta, hipComplex *C, int ldc)

hipsparseStatus_t hipsparseZbsrmm(hipsparseHandle_t handle, hipsparseDirection_t dirA, hipsparseOperation_t transA, hipsparseOperation_t transB, int mb, int n, int kb, int nnzb, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descrA, const hipDoubleComplex *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, const hipDoubleComplex *B, int ldb, const hipDoubleComplex *beta, hipDoubleComplex *C, int ldc)

/*

Sparse matrix dense matrix multiplication using BSR storage format.

hipsparseXbsrmm multiplies the scalar with a sparse matrix , defined in BSR storage format, and the dense matrix (where) and adds the result to the dense matrix (where) that is multiplied by the scalar.

*/

4.4.2 hipsparseXcsrmm

hipsparseStatus_t hipsparseScsrmm(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, int k, int nnz, const float *alpha, const hipsparseMatDescr_t descrA, const float *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const float *B, int ldb, const float *beta, float *C, int ldc)

hipsparseStatus_t hipsparseDcsrmm(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, int k, int nnz, const double *alpha, const hipsparseMatDescr_t descrA, const double *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const double *B, int ldb, const double *beta, double *C, int ldc)

hipsparseStatus_t hipsparseCcsrmm(hipsparseHandle_t handle, hipsparseOperation_t transA, int m, int n, int k, int nnz, const hipComplex *alpha, const hipsparseMatDescr_t descrA, const hipComplex *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const hipComplex *B, int ldb, const hipComplex *beta, hipComplex *C, int ldc)

hipsparseStatus_t hipsparseZcsrmm(hipsparseHandle_t handle, hipsparseOperation_t

transA, int m, int n, int k, int nnz, const hipDoubleComplex *alpha, const hipsparseMatDescr_t descrA, const hipDoubleComplex *csrSortedValA, const int *csrSortedRowPtrA, const int *csrSortedColIndA, const hipDoubleComplex *B, int ldb, const hipDoubleComplex *beta, hipDoubleComplex *C, int ldc)

/*

Sparse matrix dense matrix multiplication using CSR storage format.

hipsparseXcsrmm multiplies the scalar with a sparse matrix , defined in CSR storage format, and the dense matrix and adds the result to the dense matrix that is multiplied by the scalar.

*/

4.4.3 hipsparseXgemmi

hipsparseStatus_t hipsparseSgemmi(hipsparseHandle_t handle, int m, int n, int k, int nnz, const float *alpha, const float *A, int lda, const float *cscValB, const int *cscColPtrB, const int *cscRowIndB, const float *beta, float *C, int ldc)

hipsparseStatus_t hipsparseDgemmi(hipsparseHandle_t handle, int m, int n, int k, int nnz, const double *alpha, const double *A, int lda, const double *cscValB, const int *cscColPtrB, const int *cscRowIndB, const double *beta, double *C, int ldc)

hipsparseStatus_t hipsparseCgemmi(hipsparseHandle_t handle, int m, int n, int k, int nnz, const hipComplex *alpha, const hipComplex *A, int lda, const hipComplex *cscValB, const int *cscColPtrB, const int *cscRowIndB, const hipComplex *beta, hipComplex *C, int ldc)

hipsparseStatus_t hipsparseZgemmi(hipsparseHandle_t handle, int m, int n, int k, int nnz, const hipDoubleComplex *alpha, const hipDoubleComplex *A, int lda, const hipDoubleComplex *cscValB, const int *cscColPtrB, const int *cscRowIndB, const hipDoubleComplex *beta, hipDoubleComplex *C, int ldc)

/*

Dense matrix sparse matrix multiplication using CSR storage format.

hipsparseXgemmi multiplies the scalar with a dense matrix and the sparse matrix , defined in CSR storage format and adds the result to the dense matrix that is multiplied

by the scalar.

*/

4.5 预处理器函数

4.5.1 hipsparseXbsrilu02

hipsparseStatus_t hipsparseSbsrilu02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, float *bsrSortedValA_valM, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)

hipsparseStatus_t hipsparseDbsrilu02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, double *bsrSortedValA_valM, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)

hipsparseStatus_t hipsparseCbsrilu02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, hipComplex *bsrSortedValA_valM, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)

hipsparseStatus_t hipsparseZbsrilu02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, hipDoubleComplex *bsrSortedValA_valM, const int *bsrSortedRowPtrA, const int *bsrSortedColIndA, int blockDim, bsrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)

/*

Incomplete LU factorization with 0 fill-ins and no pivoting using BSR storage format.

hipsparseXbsrilu02 computes the incomplete LU factorization with 0 fill-ins and no pivoting of a sparse BSR matrix.

hipsparseXbsrilu02 requires a user allocated temporary buffer. Its size is returned by hipsparseXbsrilu02_bufferSize(). Furthermore, analysis meta data is required. It can be obtained by hipsparseXbsrilu02_analysis(). hipsparseXbsrilu02 reports the first zero pivot (either numerical or structural zero). The zero pivot status can be obtained by

```
calling hipsparseXbsrilu02_zeroPivot().
*/
```

4.5.2 hipsparseXcsrilu02

```
hipsparseStatus_t hipsparseScsrilu02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, float *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseDcsrilu02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, double *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseCcsrilu02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, hipComplex *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseZcsrilu02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, hipDoubleComplex *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csrilu02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
/*
Incomplete LU factorization with 0 fill-ins and no pivoting using CSR storage format.
hipsparseXcsrilu02 computes the incomplete LU factorization with 0 fill-ins and no pivoting of a sparse CSR matrix.
hipsparseXcsrilu02 requires a user allocated temporary buffer. Its size is returned by hipsparseXcsrilu02_bufferSize() or hipsparseXcsrilu02_bufferSizeExt(). Furthermore, analysis meta data is required. It can be obtained by hipsparseXcsrilu02_analysis(). hipsparseXcsrilu02 reports the first zero pivot (either numerical or structural zero). The zero pivot status can be obtained by calling hipsparseXcsrilu02_zeroPivot().
*/
```

4.5.3 hipsparseXbsric02

```
hipsparseStatus_t hipsparseSbsric02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, float *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, bsric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseDbsric02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, double *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, bsric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseCbsric02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, hipComplex *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, bsric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseZbsric02(hipsparseHandle_t handle, hipsparseDirection_t dirA, int mb, int nnzb, const hipsparseMatDescr_t descrA, hipDoubleComplex *bsrValA, const int *bsrRowPtrA, const int *bsrColIndA, int blockDim, bsric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
/*
Incomplete Cholesky factorization with 0 fill-ins and no pivoting using BSR storage format.
hipsparseXbsric02 computes the incomplete Cholesky factorization with 0 fill-ins and no pivoting of a sparse BSR matrix.
hipsparseXbsric02 requires a user allocated temporary buffer. Its size is returned by hipsparseXbsric02_bufferSize(). Furthermore, analysis meta data is required. It can be obtained by hipsparseXbsric02_analysis(). hipsparseXbsric02 reports the first zero pivot (either numerical or structural zero). The zero pivot status can be obtained by calling hipsparseXbsric02_zeroPivot().
*/
```

4.5.4 hipsparseXcsric02

```
hipsparseStatus_t hipsparseScsric02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, float *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseDcsric02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, double *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseCcsric02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, hipComplex *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
hipsparseStatus_t hipsparseZcsric02(hipsparseHandle_t handle, int m, int nnz, const hipsparseMatDescr_t descrA, hipDoubleComplex *csrSortedValA_valM, const int *csrSortedRowPtrA, const int *csrSortedColIndA, csric02Info_t info, hipsparseSolvePolicy_t policy, void *pBuffer)
/*
Incomplete Cholesky factorization with 0 fill-ins and no pivoting using CSR storage format.
hipsparseXcsric02 computes the incomplete Cholesky factorization with 0 fill-ins and no pivoting of a sparse CSR matrix.
hipsparseXcsric02 requires a user allocated temporary buffer. Its size is returned by hipsparseXcsric02_bufferSize() or hipsparseXcsric02_bufferSizeExt(). Furthermore, analysis meta data is required. It can be obtained by hipsparseXcsric02_analysis(). hipsparseXcsric02 reports the first zero pivot (either numerical or structural zero). The zero pivot status can be obtained by calling hipsparseXcsric02_zeroPivot().
*/
```

4.5.5 hipsparseXgtsv2

```
hipsparseStatus_t hipsparseSgtsv2(hipsparseHandle_t handle, int m, int n, const float *dl, const float *d, const float *du, float *B, int ldb, void *pBuffer)
hipsparseStatus_t hipsparseDgtsv2(hipsparseHandle_t handle, int m, int n, const double *dl, const double *d, const double *du, double *B, int ldb, void *pBuffer)
hipsparseStatus_t hipsparseCgtsv2(hipsparseHandle_t handle, int m, int n, const hipComplex *dl, const hipComplex *d, const hipComplex *du, hipComplex *B, int ldb, void *pBuffer)
/*
hipsparseStatus_t hipsparseZgtsv2(hipsparseHandle_t handle, int m, int n, const hipDoubleComplex *dl, const hipDoubleComplex *d, const hipDoubleComplex *du, hipDoubleComplex *B, int ldb, void *pBuffer)
Tridiagonal solver with pivoting.
hipsparseXgtsv2 solves a tridiagonal system for multiple right hand sides using pivoting.
*/
```

5 使用指南

5.1 设备管理

在启动 HIP 内核之前，可以调用 hipSetDevice()以设置设备。如果不调用该函数，系统将使用默认设备。除非显式调用指定其他设备，否则 HIP 内核始终在设备 0 上启动。 这是一种 HIP 设备管理方法，并非特定于 hipSPARSE 库。hipSPARSE 遵循这种方法，并假设在 hipSPARSE 例程调用之前已经设置了首选设备。

设置设备后，可以使用 hipsparseCreate()创建句柄。 后续的 hipSPARSE 例程将此句柄作为输入参数。hipSPARSE 仅查询指定的设备。如果 hipSPARSE 没有看到有效的设备，它会返回一条错误消息。需要向 hipSPARSE 提供有效的设备并确保设备安全。

如果要更改设备，则必须使用 hipsparseDestroy()销毁当前句柄，并使用

hipsparseCreate()创建另一个句柄并指定另一个设备。。

5.2 流管理

HIP 内核始终在队列（也称为流）中启动。如果未显式指定流，系统将提供并维护默认流。您无法创建或销毁默认流。但是，您可以自由创建新流，并使用 hipsparseSetStream()将其绑定到 hipSPARSE 句柄。HIP 内核在 hipSPARSE 例程中调用。hipSPARSE 句柄始终与流相关联，hipSPARSE 将其流传递给例程内的内核。一个 hipSPARSE 例程在一次调用中仅接受一个流。如果您创建了流，则需要再使用后销毁它

5.3 异步执行

除了内部具有防止异步性的内存分配函数外，除非另有说明，否则所有 hipSPARSE 库函数都是非阻塞的，并且相对于主机异步执行。该函数可能会在实际计算完成之前返回。要强制同步，使用 hipDeviceSynchronize() 或 hipStreamSynchronize() 。这将确保在设备或特定流中之前执行的所有 hipSPARSE 函数都已完成。

5.4 单元测试

使用以下命令运行示例程序：

```
cd [BUILD_DIR]/example
./example-csrmv 1000
```

使用以下命令进行单元测试：

```
cd [BUILD_DIR]/clients/tests
./hipsparse-test
```

要运行特定测试，请使用 --gtest_filter=match，其中 match 是通配符模式的 ':' 分隔列表（称为正模式），后跟一个 '-' 和另一个 ':' 分隔的模式列表。例如，使用以下命令运行 coo2csr 测试：

```
cd [BUILD_DIR]/clients/tests
./hipsparse-test --gtest_filter=*coo2csr*
```

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区